



Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251235
Nama Lengkap	NATALIA GRECIA PUTRI
Minggu ke / Materi	09 / Pengolahan String dan Regex Expression

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Pengantar String

String merupakan salah satu tipe data yang paling sering digunakan dalam pemrograman. Secara sederhana, string adalah rangkaian karakter yang tersusun menjadi satu kesatuan nilai. Karakter-karakter dalam string dapat berupa huruf, angka, maupun simbol khusus. Setiap karakter dalam string disimpan berdasarkan kode ASCII. Perlu dicatat bahwa tidak semua bahasa pemrograman memiliki string sebagai tipe data bawaan, dalam bahasa C misalnya, string tidak tersedia secara langsung karena pada dasarnya string terdiri lebih dari satu karakter. Di berbagai bahasa pemrograman lain, string juga dikenal dengan sebutan array karakter atau daftar karakter.

Beberapa contoh string:

- “Halo World”: string yang terdiri dari 10 karakter termasuk spasi.
- “12345”: string yang memuat 5 karakter angka.
- “@#&*!”: string yang memuat karakter-karakter simbol.

Pengaksesan String dan Manipulasi String

Membuat string dalam program dapat dilakukan dengan mudah, cukup dengan menyimpannya ke dalam sebuah variabel. Setelah disimpan, string dapat diakses secara menyeluruh atau diambil karakter per karakter menggunakan nomor indeks. Indeks dalam string dimulai dari angka 0, sama seperti pada tipe data list, dan harus berupa bilangan bulat.

Secara internal, komputer menyimpan string sebagai daftar karakter yang berurutan, dimana setiap karakter memiliki posisi indeks yang masing-masing. Sebagai ilustrasi, string “banana” disimpan dengan indeks sebagai berikut:

b	a	n	a	n	a
[0]	[1]	[2]	[3]	[4]	[5]

Contoh:

```
buah = "mangga"

print(buah[0])
print(buah[2])
print(buah[5])

buah_lain = "apel"
hasil = buah + " dan " + buah_lain
print(hasil)

potongan = buah[1:4]
print(potongan)

huruf_besar = buah.upper()
print(huruf_besar)

panjang = len(buah)
print(panjang)
```

Output:

```
m
n
a
mangga dan apel
ang
MANGGA
6
```

Operator dan Metode String

a. OPERATOR in

Operator in berfungsi untuk memeriksa keberadaan suatu substring di dalam sebuah string. Hasilnya berupa nilai boolean, yaitu True apabila substring ditemukan, dan False apabila tidak ditemukan.

Selain operator in, string juga mendukung operator perbandingan seperti >, < dan ==. Perbandingan dilakukan berdasarkan urutan leksikografis yang mengacu pada nilai ASCII masing-masing karakter.

Contoh:

```
print("apel" > "anggur")
print("123" == "123")
print("abc" != "ABC")

if "kucing" < "anjing":
    print("Kucing lebih kecil dalam urutan kamus")
else:
    print("Anjing lebih kecil dalam urutan kamus")
```

Output:

```
True
True
True
Anjing lebih kecil dalam urutan kamus
```

b. FUNGSI len

Fungsi len() digunakan untuk mengetahui jumlah karakter dalam sebuah string. Karena indeks string dimulai dari 0, indeks karakter terakhir dapat diakses dengan len(string)-1.

Contoh:

```
teks = "belajar python"

print(len(teks))

print(teks[len(teks) - 1])
print(teks[-1])

print(teks[-2])
```

Output:

```
14
n
n
o
```

c. TRAVERSING STRING

Traversing adalah proses membaca setiap karakter dalam string secara berurutan satu per satu. Terdapat dua pendekatan yang umum digunakan:

- Menggunakan while loop dengan bantuan variabel indeks.

```
kata = "komputer"
i = 0
while i < len(kata):
    print(kata[i], end=' ')
    i += 1
```

Output:

```
k o m p u t e r
```

- Menggunakan for loop secara langsung tanpa perlu mengelola indeks secara manual.

```
kata = "komputer"
for huruf in kata:
    print(huruf, end=' ')
```

Output:

```
k o m p u t e r
```

d. STRING SLICE

String slice adalah cara mengambil segian karakter dari sebuah string menggunakan notasi indeks awal dan akhir. Python menyediakan beberapa pola slicing yang fleksibel:

- String[awal : akhir] => mengambil karakter dari indeks awal hingga akhir.
- String[awal:] => mengambil dari indeks awal sampai akhir string.
- String[:akhir] => mengambil dari awal string hingga indeks akhir – 1.
- String[awal : akhir : langkah] => mengambil karakter dengan interval tertentu.

Penting untuk dipahami bahwa string di Python bersifat immutable, artinya isi string tidak dapat diubah secara langsung setelah dibuat. Setiap operasi yang tampaknya memodifikasi string sebenarnya menghasilkan objek string yang baru. Apabila ingin mengubah karakter tertentu, kita harus membuat variabel baru yang menampung hasil perubahannya.

Python menyediakan berbagai method bawaan untuk memudahkan pengolahan string. Karena string bersifat immutable, semua method tersebut tidak mengubah string aslinya, melainkan menghasilkan string baru. Berikut ringkasan method yang sering digunakan:

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang diinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang diinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

e. Operator * dan + pada String

Python mendukung penggunaan dua operator aritmatika untuk manipulasi string dengan cara yang khusus:

- Operator "+" digunakan untuk menggabungkan (concatenate) dua string atau lebih menjadi satu.
- Operator "*" digunakan untuk mengulang (repeat) sebuah string sebanyak jumlah yang ditentukan.

Parsing String

Parsing string adalah proses memecahkan atau menguraikan sebuah string untuk mengekstrak bagian-bagian tertentu yang dibutuhkan. Teknik ini sangat berguna ketika kita perlu mengubah format data yang ada di dalam string, misalnya mengambil informasi tanggal dari sebuah kalimat dan mengonversinya ke format yang berbeda.

Sebagai contoh, apabila terdapat tanggal dalam format YYYY-MM-DD di dalam kalimat, kita dapat menggunakan metode `split()` untuk memecah string tersebut dan kemudian menyusun ulang komponennya ke format DD/MM/YYYY.

Pengantar Regex

Regular Expression, atau yang biasa disingkat Regex, adalah sebuah mekanisme pencarian dan manipulasi teks yang bekerja berdasarkan pola-pola tertentu. Regex sangat berguna dalam berbagai skenario, di antaranya:

- Mendeteksi keberadaan pola tertentu di dalam teks.
- Mengekstrak data spesifik dari string yang kompleks.
- Menvalidasi format input seperti alamat email atau nomor telepon.
- Mengganti bagian-bagian teks berdasarkan pola yang cocok.

Dibandingkan operasi string biasa, regex menawarkan fleksibilitas yang jauh lebih tinggi dan mampu menangani pola teks yang kompleks secara efisien, terutama saat memproses teks dalam jumlah besar.

Python menyediakan modul `re` sebagai antarmuka untuk menggunakan regex. Terdapat tiga fungsi utama yang paling sering dipakai:

1. `re.search()` -> mencari kemunculan pertama dari pola yang diinginkan dalam teks.
2. `re.findall()` -> mengumpulkan semua kemunculan pola dalam bentuk daftar.
3. `re.sub()` -> mengganti bagian teks yang cocok dengan pola menggunakan teks pengganti.

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Library Python

Agar dapat memanfaatkan regex secara maksimal, penting untuk memahami karakter-karakter khusus (meta character) yang memiliki makna tertentu dalam pola regex. Berikut adalah penjelasan dari beberapa karakter penting:

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{ }	Karakter dengan arti khusus dan escaped character	\{ }d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
\$	Dakhiri dengan	this\$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{ }d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{ }d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{ }d+	Minimal 1 s/d tak terhingga karakter
{ }	Tepat sebanyak yang ada para { }	\{ }d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
	atau	\{ }d \{ }s	1 digit atau 1 spasi

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

https://github.com/grec12/71251235_nataliagrecia.git

```
1 import re
2
3 def cek_anagram(kata_pertama, kata_kedua):
4     kata_pertama = re.sub(r"[^a-zA-Z]", "", kata_pertama).lower()
5     kata_kedua = re.sub(r"[^a-zA-Z]", "", kata_kedua).lower()
6     return sorted(kata_pertama) == sorted(kata_kedua)
7
8 kata1 = input("Masukkan kata pertama: ")
9 kata2 = input("Masukkan kata kedua: ")
10
11 if cek_anagram(kata1, kata2):
12     print(f"'{kata1}' dan '{kata2}' adalah anagram.")
13 else:
14     print(f"'{kata1}' dan '{kata2}' bukan anagram")
```

Output:

```
Masukkan kata pertama: kasur
Masukkan kata kedua: rusak
'kasur' dan 'rusak' adalah anagram.
```

Penjelasan:

1. Penggunaan modul re: Modul re diimpor untuk keperluan manipulasi string menggunakan ekspresi reguler, khususnya untuk menghapus karakter yang bukan huruf sebelum proses perbandingan dilakukan.
2. Pembersihan karakter non-huruf: Sebelum membandingkan kedua kata, semua karakter selain huruf alfabet dihapus menggunakan `re.sub()`. Ini memastikan bahwa simbol atau angka tidak ikut memengaruhi hasil pengecekan.
3. Konversi ke huruf kecil: Kedua kata diubah seluruhnya menjadi huruf kecil agar perbandingan tidak terpengaruh oleh perbedaan kapitalisasi. Misalnya, 'kasur' dan 'rusak' akan dianggap sama.

4. Pengurutan karakter: Karakter dari setiap kata diurutkan secara alfabetis menggunakan `sorted()`. Jika hasil pengurutan kedua kata identik, maka keduanya adalah anagram.
5. Pengecekan dan output: Fungsi `cek_anagram()` dipanggil dengan dua kata dari input pengguna. Hasilnya menentukan pesan yang ditampilkan, apakah kedua kata merupakan anagram atau bukan.

SOAL 2

```
1  import re
2
3  def hitung_kata(teks, kata_dicari):
4      teks = re.sub(r"[^\w\s]", "", teks).lower()
5      kata_dicari = kata_dicari.lower()
6      list_kata = teks.split()
7      total = list_kata.count(kata_dicari)
8      return total
9
10 kal = input("Masukkan kalimat yang diinginkan: ")
11 kata = input("Masukkan kata yang ingin dihitung: ")
12
13 frekuensi = hitung_kata(kal, kata)
14 print(f"'{kata}' dan {frekuensi} buah.")
```

Output:

```
Masukkan kalimat yang diinginkan: Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan
Masukkan kata yang ingin dihitung: makan
'makan' dan 3 buah.
```

Penjelasan:

1. Penghapusan tanda baca: Fungsi `re.sub()` dengan pola `[^\w\s]` digunakan untuk membuang semua karakter yang bukan huruf, angka, atau spasi dari teks. Langkah ini memastikan tanda baca tidak mengganggu proses penghitungan kata.
2. Penyeragaman huruf: Baik teks maupun kata yang ingin dicari diubah ke huruf kecil semua, sehingga pencocokan tidak peka terhadap kapitalisasi.
3. Pemecahan teks menjadi daftar kata: Metode `split()` memisahkan teks menjadi daftar berdasarkan spasi, menghasilkan kumpulan kata individual.

4. Penghitungan kemunculan: Metode count() menghitung berapa kali kata yang dicari muncul dalam daftar tersebut.

Input dan output: Program meminta pengguna memasukkan kalimat dan kata target, lalu menampilkan hasil penghitungan dalam format yang mudah dibaca.

SOAL 3

```
1 import re
2
3 def hapus_spasi_berlebih(teks):
4     teks = re.sub(r"\s+", " ", teks)
5     teks = teks.strip()
6     return teks
7
8 kalimat = input("Masukkan kalimat yang ingin dihapus spasi yang berlebih: ")
9 hasil = hapus_spasi_berlebih(kalimat)
10 print(f"Hasil: '{hasil}'")
```

Output:

```
Masukkan kalimat yang ingin dihapus spasi yang berlebih: saya tidak suka memancing ikan
Hasil: 'saya tidak suka memancing ikan'
```

Penjelasan

1. Input pengguna: Pengguna diminta memasukkan kalimat yang mungkin mengandung spasi ganda atau lebih. Kalimat tersebut disimpan ke dalam variabel kalimat.
2. Penggantian spasi berlebih: Di dalam fungsi hapus_spasi_berlebih(), ekspresi reguler \s+ digunakan untuk mendeteksi satu atau lebih spasi berturut-turut dan menggantinya dengan satu spasi tunggal.
3. Pembersihan awal dan akhir: Metode strip() kemudian digunakan untuk membuang spasi yang tersisa di bagian paling awal dan paling akhir teks.
4. Pengembalian dan tampilan hasil: Fungsi mengembalikan teks yang sudah bersih, lalu program mencetak hasilnya ke layar.

SOAL 4

```
1  import re
2
3  def cari_kata_terpendek_terpanjang(kalimat):
4      kata_kata = re.findall(r"\b\w+\b", kalimat)
5
6      if not kata_kata:
7          return None, None #Tidak ada kata ditemukan
8      kata_pendek = kata_panjang = kata_kata[0]
9
10     for kata in kata_kata:
11         if len(kata) < len(kata_pendek):
12             kata_pendek = kata
13         if len(kata) > len(kata_panjang):
14             kata_panjang = kata
15
16     return kata_pendek, kata_panjang
17
18 kalimat = input("Masukan kalimat yang ingin anda cek: ")
19 terpendek, terpanjang = cari_kata_terpendek_terpanjang(kalimat)
20
21 print(f"Terpendek: {terpendek}")
22 print(f"Terpanjang: {terpanjang}")
```

Masukan kalimat yang ingin anda cek: red snakes and a black frog in the pool
Terpendek: a
Terpanjang: snakes

Penjelasan:

1. Input pengguna: Pengguna memasukkan kalimat yang ingin diperiksa.
2. Ekstraksi kata dengan regex: Fungsi `re.findall()` dengan pola `\b\w+\b` digunakan untuk mengambil semua kata dalam kalimat. Pola ini memastikan hanya rangkaian karakter alfanumerik yang dihitung sebagai kata, sehingga tanda baca tidak ikut terambil.
3. Penanganan kalimat kosong: Jika tidak ada kata yang ditemukan, fungsi langsung mengembalikan `None` untuk keduanya sebagai penanganan kondisi tepi.
4. Inisialisasi nilai awal: Variabel `kata_pendek` dan `kata_panjang` diisi dengan kata pertama yang ditemukan sebagai titik pembanding awal.
5. Iterasi dan perbandingan: Program menelusuri setiap kata dan membandingkan panjangnya. Jika sebuah kata lebih pendek dari `kata_pendek` saat ini, maka kata tersebut menjadi `kata_pendek` baru. Begitu pula sebaliknya untuk `kata_panjang`.
6. Tampilan hasil: Kata terpendek dan terpanjang dicetak ke layar setelah iterasi selesai.

SOAL 5

```
1 import re
2 from datetime import datetime
3
4 def cari_dan_hitungan_selisih(teks):
5     daftar_tanggal = re.findall(r"\b\d{4}-\d{2}-\d{2}\b", teks)
6     sekarang = datetime.now()
7
8     for tgl_str in daftar_tanggal:
9         tgl_obj = datetime.strptime(tgl_str, "%Y-%m-%d")
10        selisih = (sekarang - tgl_obj).days
11        print(f"{tgl_obj} selisih {selisih} hari")
12
13 teks = "Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan " \
14 "nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan Ki "
15 "Hajar Dewantara (1889-05-02)."
```

Output:

```
1945-08-17 00:00:00 selisih 29464 hari
1785-11-11 00:00:00 selisih 87816 hari
1783-06-08 00:00:00 selisih 88703 hari
1889-05-02 00:00:00 selisih 50024 hari
```

Penjelasan:

1. Pengambilan tanggal dari teks: `re.findall()` dengan pola `\b\d{4}-\d{2}-\d{2}\b` digunakan untuk mendeteksi semua tanggal berformat YYYY-MM-DD dalam teks. Seluruh tanggal yang berhasil ditemukan disimpan dalam daftar `daftar_tanggal`.
2. Pengambilan tanggal sekarang: `datetime.now()` menghasilkan objek `datetime` yang merepresentasikan waktu saat program dijalankan, digunakan sebagai acuan perhitungan selisih.
3. Iterasi dan konversi tanggal: Setiap tanggal dalam `list_tanggal` dikonversi dari bentuk string menjadi objek `datetime` menggunakan `datetime.strptime()` dengan format `%Y-%m-%d`.
4. Penghitungan selisih hari: Selisih antara tanggal sekarang dan tanggal historis dihitung dengan operator pengurangan antar objek `datetime`, lalu diambil nilainya dalam satuan hari melalui properti `.days`.

5. Tampilan hasil: Setiap tanggal ditampilkan bersama jumlah selisih harinya dalam satu baris output.

SOAL 6

```
1  import re
2  import random
3  import string
4
5  def random_password(length=8):
6      karakter = string.ascii_letters + string.digits
7      return "".join(random.choice(karakter) for _ in range(length))
8
9  def process_emails(teks):
10     polaEmail = r"([a-zA-Z0-9.%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})"
11     emails = re.findall(polaEmail, teks)
12     hasil = []
13     for email in emails:
14         username = email.split('@')[0]
15         password = random_password()
16         hasil.append(f"{email} username: {username}, password: {password}")
17     return "\n".join(hasil)
18
19     teks = """
20     Berikut adalah daftar email dan nama pengguna dari mailing list:
21     anton@mail.com dimiliki oleh antonius
22     budi@gmail.co.id dimiliki oleh budi anwari
23     slamet@getnada.com dimiliki oleh slamet slumut
24     matahari@tokopedia.com dimiliki oleh toko matahari
25     """
26
27     output = process_emails(teks)
28     print(output)
```

```
anton@mail.com username: anton, password: M3aeCVF2
budi@gmail.co.id username: budi, password: DANVbxoT
budi@gmail.co.id username: budi, password: DANVbxoT
slamet@getnada.com username: slamet, password: 9RKcXkbf
matahari@tokopedia.com username: matahari, password: 3CfxqtUS
```

Penjelasan:

1. Pembuatan password acak (random_password): Fungsi ini menghasilkan string acak sepanjang 8 karakter yang terdiri dari kombinasi huruf (kapital dan kecil) serta angka. Setiap karakter dipilih secara acak menggunakan random.choice() dari kumpulan karakter yang tersedia.
2. Ekstraksi email dari teks (process_emails): Fungsi ini menjalankan dua tugas utama. Pertama, pola regex digunakan untuk menemukan semua alamat email yang valid dalam teks. Pola tersebut mendeteksi format standar email yang terdiri dari bagian lokal, simbol @, nama domain, dan ekstensi domain.

3. Pengambilan username: Untuk setiap email yang ditemukan, bagian sebelum tanda @ diambil menggunakan `split('@')[0]` sebagai username pengguna.
4. Pembuatan password: Fungsi `generate_random_password()` dipanggil untuk setiap email guna menghasilkan password unik dan acak.
5. Penggabungan dan tampilan hasil: Setiap baris hasil yang memuat email, username, dan password digabungkan menjadi satu string menggunakan pemisah baris baru (`\n`), kemudian dicetak seluruhnya ke layar.